Online Marketplace Database System

Course: CSE 535 Database Systems

Submission Date: 15 June 2025

1. Domain and Database Design

For this project, I modeled an online marketplace system. The design captures customers, products, orders, payments, reviews, and product categories.

Table	Description
Customer	Stores customer ID, name, email, created date
Product	Contains product ID, name, price, and stock
Category	Defines product categories
ProductCategory	A junction table, I used to implement a many-to-many relationship between Product and Category
Order	Links customers to orders with date and amount
OrderItem	Item-level breakdown of each order
Payment	Payment record linked to orders
Review	Customer product reviews with rating, comment, and date

Table 1: Schema overview

I intentionally kept the default behavior (ON DELETE NO ACTION, ON UPDATE NO ACTION) in all foreign key constraints to maintain manual control over deletion and updates. I wanted to prevent accidental data loss and ensure that all deletions as well as updates are handled manually and explicitly.

1.1 Normalization

The database I designed follows third normal form (3NF), which means that each table stores only one type of information. There is no duplicate or unnecessary data. Every piece of data is stored in the right place, without depending on anything that doesn't uniquely identify it. I used foreign keys to link related tables, so the relationships between them are clear and safe. Every table has a primary key to make sure each row is unique.

2. Data Generation

I generated data using the Faker Python library of a total over 1.43 million rows.

Table	Rows
Customer	100,000
Product	10,000
Category	10
ProductCategory	20000
Order	300,000
OrderItem	600,000
Payment	300,000
Review	100,000

Table 2: Number of records generated per table

3. Baseline Querying

I wrote nineteen baseline analytical SQL queries to reflect realistic business needs. Those include:

- Joins across multiple related tables
- Aggregations and grouping
- Subqueries and Having clauses
- Trend analysis
- Self join, cross join
- Exists

Sample questions answered:

1. Who are the most frequent customers?
2. Which products are most sold by quantity?
3. What are the most reviewed products?
4. What is the average product rating per category?
5. What are the monthly trends in order volume and payments?
6. Which products have never been sold?
7. Which products generated the highest total revenue?
8. Which products share the same price?

4. Benchmarking Results

I executed each of the 19 baseline queries 10 times. Execution time per run was measured using Python's time module. For each query, I collected the mean and standard deviation of the runtimes. This benchmarking helps identify expensive queries. It forms the foundation for future optimizations in project 2.

Query	Avg time (s)	Std dev (s)
Products with same price	16.4477	0.5241
Most frequently bought product pair	4.3231	0.1703
Customers who ordered most expensive product	1.8163	0.0876
Most sold products	1.3845	0.1439
Highest spending customers	1.9905	0.1904
Products with same price	0.7710	0.1283

Table 3: Benchmark of 6 sample queries

4.1 Observation

Most queries completed under 0.5 seconds, especially those involving simple GROUP BY, JOIN, and filters. From table 3, we can see that some queries involving large joins, self-joins, or pairwise comparisons took longer, reaching up to 16.4 seconds.

5. Files Included

1. marketplace_schema.sql: Defines schema, constraints, and keys.
2. data_generation.py: Python data generation script using Faker.
3. baseline_marketplace_query.sql: All SQL queries I used.
4. benchmark_results.csv: Benchmark results with average and std dev.
5. benchmarking.py: Benchmarking script.
6. This summary report.